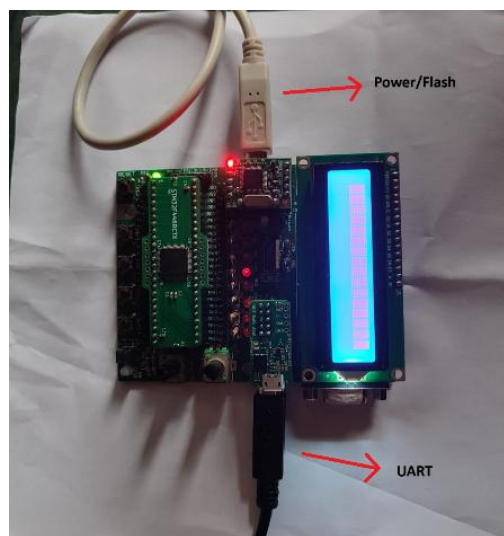


UART/USART Hardware Setup

Connecting the UCT STM32F4 board to a PC serial terminal



The completed arrangement uses one USB connection for power/programming and a second connection for UART communication.

PURPOSE	BOARD	SERIAL FORMAT	OUTCOME
First UART link	STM32F051	115200, 8N1	Terminal output

Tutorial objective

By the end of this activity, you should be able to assemble the STM32-to-PC UART hardware, configure USART in STM32CubeIDE, write and flash a simple transmit program, configure a PC terminal and verify a repeatable UART transmission.

Before you begin

UART is an asynchronous serial interface: the two devices do not share a clock line, so both ends must use matching communication settings. In this tutorial the STM32 transmits through the STM32USB2UART adapter and the PC displays the received characters in a serial terminal.

Required equipment

- UCT STM32F051 development board;
- STM32USB2UART adapter board;
- 2×5 female header for the adapter;
- soldering iron, solder and a safe soldering workspace;
- USB-A to USB-B cable for board power and programming;
- USB-A to Micro-USB cable, or the appropriate USB-B connection, for UART;
- PC with a serial terminal such as PuTTY, Tera Term, screen or minicom; and
- STM32CubeIDE.



STM32USB2UART adapter

Board selection

Create the STM32CubeIDE project for the **STM32F051 device fitted to the course board**. Do not select an STM32F4 target or reuse an ELF file from a previous year. In this tutorial, you will generate firmware specifically for the STM32F051.

What 115200, 8N1 means

115200 baud	Both endpoints use a nominal rate of 115200 bits per second.
8 data bits	Each character contains eight data bits.
N: no parity	No parity bit is added to the frame.
1 stop bit	One stop bit terminates each character.
No flow control	RTS/CTS and software flow control are disabled.

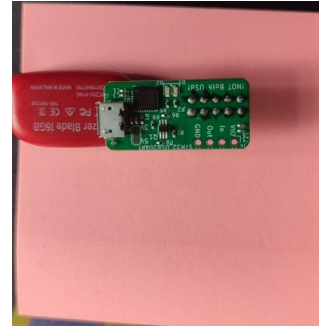
Assemble the hardware

Step 1 — Solder the adapter header

Disconnect all power. Insert the 2×5 female header into the STM32USB2UART board, confirm that it is straight and solder every pin. Inspect the joints for bridges, incomplete wetting or loose pins.



Header positioned before soldering



Adapter supported during soldering

Soldering safety

Never solder while the STM32 or adapter is connected to USB. Work in a ventilated area, keep the iron in its stand and allow the board to cool before handling it.

Step 2 — Fit the adapter to the STM32 board

Align the adapter with the STM32F051 course board's matching 2×5 connector. Check the orientation and pin labels before applying pressure; do not force the connector. A one-row or reversed offset can damage the boards.

- Confirm that adapter ground is aligned with board ground.
- Confirm that the adapter TX path reaches the STM32 UART RX pin and the adapter RX path reaches the STM32 UART TX pin.
- Ask a demonstrator to check the orientation if the connector is not keyed or clearly labelled.

Step 3 — Connect both USB cables

Connect the board's programming/power USB cable and then connect the UART adapter's USB cable. These connections serve different purposes:

- **Power/programming link:** powers the STM32 board and supports flashing/debugging.
- **UART link:** appears as a serial port on the PC and carries the transmitted characters.

Do not connect the UART adapter directly to an RS-232 voltage-level port. The course adapter uses logic-level UART on the board side.

Configure the PC serial terminal**Step 4 — Identify the UART serial port**

Windows: open Device Manager and inspect *Ports (COM & LPT)*. Disconnect and reconnect the UART cable if necessary to identify the new COM port.

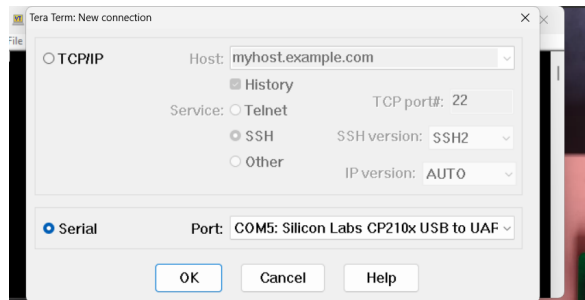
Linux: compare the serial devices before and after connecting the adapter:

```
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null  
dmesg -follow
```

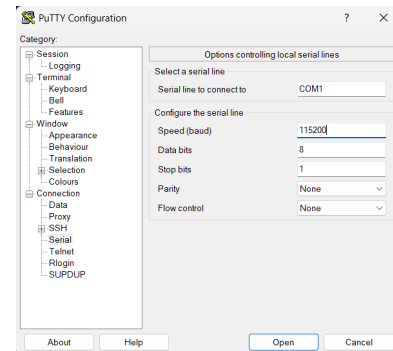
Do not assume the port number remains the same after reconnecting or moving to another USB socket.

Step 5 — Open the terminal with matching settings

Configure the terminal for **115200 baud, 8 data bits, no parity, 1 stop bit and no flow control**. Select the UART adapter's serial port, not the programming/debug port.



Tera Term serial-port selection



PuTTY serial settings

On Linux, one possible command is:

```
screen /dev/ttyUSB0 115200
```

Replace `/dev/ttyUSB0` with the device found in Step 4. To exit screen, press `Ctrl-A`, then `K`, then confirm.

If Linux reports “Permission denied”

Check the device ownership with `ls -l`. On many systems, serial access is granted through the `dialout` group. Follow the laboratory machine's access policy rather than running the terminal as root.

Create and test the firmware

Step 6 — Create the STM32F051 project

Open STM32CubeIDE and create a new STM32 project for the exact STM32F051 device on the course board. In the device configuration view:

1. enable the USART peripheral connected to the STM32USB2UART adapter in *Asynchronous* mode;
2. set the baud rate to **115200**;
3. select **8 data bits, no parity and 1 stop bit**;
4. disable hardware flow control; and
5. confirm the generated TX and RX pins against the course-board pinout before generating the code.

The handle name generated by CubeIDE may be `huart1`, `huart2`, or another name depending on the USART instance used. The example below assumes `huart1`.

Step 7 — Write the transmit loop

In Core/Src/main.c, place the following code inside the generated while (1) user-code region. Keep it between CubeIDE's USER CODE markers so that later code generation does not remove it.

```
static const uint8_t message[] =
    "Welcome to Binary\r\n";

HAL_UART_Transmit(&huart1,
    (uint8_t *)message,
    sizeof(message) - 1,
    HAL_MAX_DELAY);

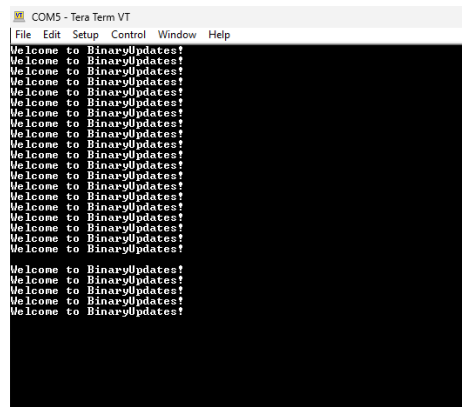
HAL_Delay(1000);
```

If CubeIDE generated a different UART handle, replace huart1 with that handle. Build the project, resolve all compiler errors, connect the programmer and use *Run* or *Debug* to flash the STM32F051.

Step 8 — Verify UART transmission

Keep the serial terminal open and reset or power-cycle the STM32 board. The terminal should repeatedly display:

```
Welcome to Binary
```



If readable text appears consistently, the hardware path, baud rate and frame configuration are compatible. Record the identified port and capture a screenshot for your laboratory notes.

Troubleshooting

Work from the physical layer upward. Change one item at a time and retest.

Symptom	Likely cause	Check or corrective action
No serial port appears	Wrong cable, charge-only cable, un-powered adapter or missing driver	Reconnect the UART cable, try a known data cable and inspect Device Manager or dmesg.
Port opens but no text appears	Wrong port, firmware not running, wrong UART instance or incorrect adapter orientation	Confirm both USB connections, reset the board, check the selected USART and generated handle, then recheck adapter alignment.
Unreadable characters	Baud rate, parity, data-bit or stop-bit mismatch	Set both ends to 115200, 8N1, no flow control.
Output appears only after reset	Terminal was opened after the program transmitted	Open the terminal first, then reset the STM32.
Programming works but UART does not	Programming and UART use different USB paths, or the wrong TX/RX pins were configured	Confirm that the terminal uses the adapter's serial port and compare the CubeIDE pinout with the course-board pinout.
Linux permission denied	User lacks permission to open the device	Inspect device ownership and use the approved serial-access group or laboratory configuration.
Intermittent connection	Poor solder joint, loose header or damaged cable	Power down, inspect the header and re-test with another cable.

Checkpoint questions

Before leaving the tutorial, be able to answer the following:

1. Why are two USB connections used in this setup?
2. What does 8N1 describe, and why must the terminal and STM32 agree?
3. Why can an incorrect baud rate produce characters rather than simply producing no output?
4. How would you distinguish a firmware problem from a serial-port selection problem?
5. Why is the code placed inside CubeIDE's USER CODE markers?
6. How would you extend the program to receive commands from the PC instead of only transmitting a welcome message?

Further reading

For STM32 UART configuration, HAL polling, interrupt and DMA functions, and deeper coverage of the USART hardware, consult:

- Khaled Magdy, *STM32 UART (USART) Tutorial + Examples (DMA, Interrupt)*, DeepBlue Embedded: <https://deepbluembedded.com/stm32-usart-uart-tutorial/>
- The reference manual and HAL documentation for the exact STM32 device used in the laboratory.

Next step
Once the one-way transmission works, extend the firmware into a framed two-way interface: receive a line of text, validate it and return an explicit acknowledgement or error response. This is the communication pattern required by the Serial Game Server project.

Updated from the 2025 UART/USART tutorial prepared by Lungelo Mabaso.